POWER APPS × AI 코딩 · 실습 교재

VS Code + Claude로 Power Apps 앱 단숨에 만들기

캔버스 앱은 사실 코드(.pa.yaml + Power Fx)입니다.
AI에게 코드를 쓰게 하면 화면 여러 장짜리 앱이 몇 분 만에 나옵니다.

윤재웅 · Power Platform & AI 업무 자동화

왜 코드로 만드나

드래그 앤 드롭의 한계를 AI가 넘어 줍니다

01

반복 작업

화면마다 컨트롤을 끌어다 놓고 위치·색·글꼴을 하나씩 맞추는 데 대부분의 시간이 듭니다.

02

앱의 실체는 텍스트

Studio에서 만든 앱은 화면별 .pa.yaml 파일과 Power Fx 수식으로 저장됩니다. 사람이 읽을 수 있는 코드입니다.

03

AI가 쓰고 검증

Claude가 요구사항을 듣고 화면·컨트롤·수식을 한꺼번에 작성하고, 오류는 스스로 고칩니다.

핵심 · 사람은 "무엇을 만들지"를 설명하고 결과를 검토합니다. 반복 배치는 AI가 맡습니다.

.msapp 파일의 정체

.msapp은 앱 전체를 묶은 압축 파일입니다

```
MyApp.msapp  (zip 형식)
└ Src\
   ├ App.pa.yaml            ← 앱 설정
   ├ HomeScreen.pa.yaml     ← 화면마다 1개
   ├ DetailScreen.pa.yaml
   └ Component\             ← 컴포넌트
```

내보내기 · Studio에서 저장 ▸ **복사본 다운로드**

가져오기 · make.powerapps.com ▸ 앱 ▸ **앱 가져오기** ▸ **파일에서 (.msapp)**, 또는 Studio에서 파일 ▸ 열기 ▸ 찾아보기

파일로 묶기 · Power Platform CLI의 `pac canvas pack`으로 소스 폴더를 .msapp으로 만들 수 있습니다.

주의 · pack/unpack은 미리 보기 기능이며 Microsoft가 사용 중단(deprecated)을 예고했습니다. 실무에서는 다음 장의 공식 경로를 권장합니다.

두 가지 경로

파일로 가져오기 vs 실시간 동기화

방식	.msapp 파일 만들어 가져오기	공식 AI 코드 생성 (권장)
연결	오프라인에서 파일 생성 → 수동 가져오기	Studio 공동 작성 세션과 실시간 동기화
검증	가져올 때 오류가 나면 직접 수정	Canvas 저작 MCP 서버가 검증, AI가 자동 수정
도구	pac canvas pack (미리 보기, 중단 예고)	Canvas Apps 플러그인 + MCP 서버
적합한 용도	앱 백업·전달, 교육용 샘플 배포	새 앱 만들기, 기존 앱 수정

두 방식 모두 결과물은 같은 .pa.yaml 코드입니다. 만들 때는 실시간 동기화로, 완성본을 주고받을 때는 .msapp으로 쓰면 됩니다.

준비물

시작 전 4가지를 확인합니다

①

.NET SDK 10 이상

MCP 서버 실행에 필요합니다. 터미널에서 `dotnet --list-sdks`로 확인합니다.

②

코딩 에이전트

VS Code(Chat · Agent 모드) 또는 Claude Code. MCP와 플러그인을 지원해야 합니다.

③

Power Platform 환경

앱을 만들고 편집할 권한이 있는 환경. 처음에는 개발용 환경을 권장합니다.

④

저장된 빈 앱 + 공동 작성

Studio에서 빈 캔버스 앱을 만들어 저장하고, 설정 ▸ 업데이트 ▸ **공동 작성**을 켭니다.

중요 · 앱은 먼저 Power Apps에 존재해야 합니다. 완전히 빈 앱이어도 괜찮지만, 저장과 공동 작성 설정은 꼭 필요합니다.

실습 1 · 설치와 연결

플러그인을 깔고 Studio 세션에 연결합니다

Claude Code

```
/plugin marketplace add microsoft/power-platform-skills
/plugin install canvas-apps@power-platform-skills
/configure-canvas-mcp
```

VS Code

확장(Extensions)에서 `@agentPlugins canvas apps` 검색 ▸ Microsoft의 **Canvas Apps** 설치 ▸ Chat을 Agent 모드로 열기

1. Studio에서 앱을 열어 둔 채 브라우저 주소(URL)를 복사합니다. 탭은 끝까지 닫지 않습니다.
2. **/configure-canvas-mcp**를 실행하고 URL을 붙여 넣은 뒤 로그인합니다. 환경·앱 ID는 자동으로 읽습니다.
3. 연결 확인: **“List the available Canvas App controls”**라고 물어 컨트롤 목록이 나오면 성공입니다.

실습 2 · 말로 앱 만들기

요구사항을 설명하면 화면별 코드가 생성됩니다

프롬프트 예시

/canvas-app
재고 관리 Canvas App을 만들어 줘.
① 품목을 검색하는 목록 화면
② 선택한 품목의 상세 화면
③ 입고·출고를 기록하는 입력 폼
데이터는 Dataverse의 '재고' 테이블을 쓰고, 남색과 주황색 기업형 테마로.

"Canvas App"이라는 말을 넣으면 에이전트가 맞는 스킬을 고릅니다. 화면 스케치 이미지를 첨부해도 됩니다.

1. AI가 사용할 수 있는 컨트롤·데이터를 MCP로 확인하고, 모호한 부분을 되묻습니다.
2. 화면마다 .pa.yaml 파일을 생성합니다.
3. MCP 서버로 검증하고 오류를 스스로 고칩니다.
4. Studio에 실시간으로 반영됩니다. 미리 보기로 바로 테스트합니다.
5. “대기 중인 항목만 보이게 필터 추가해 줘”처럼 말로 계속 고칩니다.

생성되는 코드

.pa.yaml은 이렇게 생겼습니다

```
Screens:
  HomeScreen:
    Properties:
      Fill: =RGBA(247, 246, 242, 1)
    Children:
      - TitleLabel:
          Control: Label
          Properties:
            Text: ="재고 현황"
            Size: =28
      - CountLabel:
          Control: Label
          Properties:
            Text: ="품목 " & CountRows('재고') & "개"
```

Screens 아래에 화면, **Children** 아래에 컨트롤이 들어갑니다.

Control은 컨트롤 종류, **Properties**는 속성입니다.

=로 시작하는 값은 Power Fx 수식입니다. Excel 수식처럼 데이터 개수를 세고, 화면을 이동(Navigate)하고, 데이터를 저장(Patch)합니다.

Studio의 컨트롤 ▸ **코드 보기**로 지금 만든 앱의 YAML을 확인하고 복사·붙여넣기 할 수도 있습니다.

잘 쓰는 법과 보안

작게 시작하고, 사람이 검토합니다

잘 쓰기

작게 → 크게

기본 화면부터 만들고 기능을 하나씩 더합니다. 큰 변경 후에는 바로 미리 보기로 테스트합니다. 레이아웃·색상 방향을 구체적으로 말할수록 결과가 좋습니다.

검증

생성 코드는 내 책임

AI는 최선을 다해 작성하지만, 회사 기준·접근성·보안 요건 충족 여부는 사람이 확인합니다. 개발 환경에서 먼저 검증한 뒤 운영 앱에 적용합니다.

보안

권한과 정책

신뢰할 수 있는 마켓플레이스에서만 설치하고, 필요한 권한만 가진 계정으로 연결합니다. 프롬프트에 비밀번호·토큰을 넣지 않고, 회사의 DLP(데이터 손실 방지) 정책을 따릅니다.

정리

설명은 사람이, 배치는 AI가,
검토는 다시 사람이.

캔버스 앱이 코드로 저장되기 때문에 가능한 방식입니다. 시민 개발자는 업무 설명에 집중하고, 반복적인 화면 작업은 AI에게 맡깁니다.

출처 · Microsoft Learn: Create and edit canvas apps with AI code generation tools / View source code files for canvas apps / pac canvas (Power Platform CLI) / Export and import a canvas app as a single app file